AI-Based Smart Helmet Detection System Using Deep Learning

Complete Technical Report

Institution: GSFC University · **Status:** complete (Phases 1–3) · **Report date:** 20 October 2026

This report describes everything used to build the project: the data, the techniques, the trained models, the AI pipeline, the web application, the testing, and **where the code for each part is**. All paths are relative to the project folder `D:\Helmet Detection\`.

Contents

1. Project summary
2. System architecture
3. Technology stack
4. Project folder map (where the code is)
5. Phase 1: data collection and preparation
6. Phase 1: model training
7. Phase 1: results
8. Phase 2: AI pipeline
9. Phase 3: web application
10. Testing and evaluation
11. Problems found and how they were solved
12. Limitations
13. Future work
14. How to run the project
15. Glossary

---

1. Project summary

Problem. Two-wheeler riders without helmets are a leading cause of road deaths in India. Checking them manually from traffic cameras is slow and does not scale.

Solution. The system analyses a traffic image or video and, for every motorcycle, decides:

- **VIOLATION:** at least one rider (driver or pillion) has no helmet,
- **SAFE:** every visible rider wears a helmet,
- **UNKNOWN:** a motorcycle with no visible head (the system never guesses).

For every violation it **locates and reads the number plate** with OCR, saves evidence images, and writes a JSON violation report. A simple web page lets anyone upload an image or video and see the results.

Key numbers

Item	Result
Helmet model (test mAP50, unseen Indian images)	**89.1 %**
Number-plate model (test mAP50)	**92.3 %**
Full system, image-level verdict on 157 held-out demo images	**93 %** correct (146 / 157)
Weather robustness (synthetic fog / blur / low light)	100 % / 100 % / 100 % correct verdicts
Processing speed (laptop CPU, no GPU)	about 2–6 s per image; 4K video about 13 s per video-second

---

2. System architecture

```
PHASE 3 · Web app (browser)
  upload image / video → progress → results, tables, downloads
         │
         │  HTTP (FastAPI)
         ▼
PHASE 2 · AI pipeline (each image / video frame)
  quality check → enhancement only when needed (fog, blur, noise, contrast, low light)
  → COCO YOLO26-m: motorcycles + persons     → helmet model (Phase 1): head boxes
  → association: head → rider → motorcycle   → status: VIOLATION / SAFE / UNKNOWN
  → VIOLATION: crop motorcycle (+15 %) → plate model (Phase 1) → PaddleOCR → Indian format
  → annotated image / video + evidence crops + JSON report
  video: 5 analysed frames per second, ByteTrack tracking, majority voting per motorcycle
         ▲
         │  trained models
         │
PHASE 1 · Training on Kaggle (2 × NVIDIA T4)
  11 public datasets → cleaning → duplicate removal → leak-free split → augmentation
  → Model A: helmet / no_helmet (YOLO26-m, 960 px)
  → Model B: number_plate on motorcycle crops (YOLO26-m, 640 px)
```

Three models are used:

Model	What it detects	Trained by us?	File
COCO YOLO26-m	motorcycle (COCO class 3), person (class 0)	No (pretrained on COCO)	`models/coco/yolo26m.pt`
Model A: Helmet	`helmet`, `no_helmet` head boxes	**Yes**	`models/helmet/helmet_best.pt` (+ `.onnx`)
Model B: Plate	`number_plate` on a motorcycle crop	**Yes**	`models/plate/plate_crop_best.pt` (+ `.onnx`)

Plus **PaddleOCR PP-OCRv5** (pretrained) to read the plate text.

---

3. Technology stack

Area	Technology	Version	Used for
Language	Python	3.11	everything except the web page
Deep learning	PyTorch	2.x	model training and inference
Detector framework	Ultralytics	8.4.x	YOLO26 training, validation, export, tracking

Area	Technology	Version	Used for
Detector	YOLO26-m	–	all three detection models (NMS-free, end-to-end)
OCR	PaddleOCR (PP-OCRv5 mobile), PaddlePaddle	3.7 / 3.3	reading number plates
Image processing	OpenCV, NumPy	4.10	enhancement, drawing, video I/O
Tracking	ByteTrack (in Ultralytics)	–	following each motorcycle across video frames
Video encoding	imageio-ffmpeg (H.264)	–	output videos that play in any browser
Backend API	FastAPI, Uvicorn	0.143	REST API and web server
Frontend	HTML, CSS, JavaScript (no framework)	–	upload page and results
Testing	pytest, httpx; Playwright (UI)	–	unit, end-to-end and API tests
Training hardware	Kaggle, 2 × NVIDIA T4 (16 GB each)	–	Phase 1
Run hardware	Laptop, AMD Ryzen 5 5500U, no GPU	–	Phases 2–3 (GPU used automatically if present)
Data tools	pandas, imagehash-style pHash/dHash (own implementation), Roboflow REST API, Kaggle API	–	data preparation

4. Project folder map (where the code is)

```
D:\Helmet Detection\
├── phase1\                          PHASE 1 · data preparation and model training (runs on Kaggle)
│   ├── src\data_prep.py             Notebook 1: download → clean → de-duplicate → split → augment → EDA
│   ├── src\train.py                 Notebook 2: train → evaluate → per-source test → threshold → export
│   ├── notebooks\01_data_prep.ipynb   generated from src\data_prep.py (uploaded to Kaggle)
│   ├── notebooks\02_train.ipynb       generated from src\train.py (uploaded to Kaggle)
│   ├── tools\build_notebooks.py     converts src\*.py into the .ipynb notebooks
│   ├── tools\build_demo_dataset.py  builds the categorised demo test set (demo_dataset\)
│   ├── RESULTS.md                   Phase 1 results report
│   └── README.md                    Phase 1 how-to and theory
├── models\                          trained weights (output of Phase 1)
│   ├── helmet\helmet_best.pt/.onnx            Model A (final)
│   ├── plate\plate_crop_best.pt/.onnx         Model B (final, v3)
│   ├── plate\round1_crop\                     Model B v2 (experiment, kept for comparison)
│   ├── plate\baseline_fullimage\              Model B v1 (experiment, kept for comparison)
│   └── coco\yolo26m.pt                        pretrained COCO detector
├── backend\                         PHASE 2 + 3 · AI pipeline, API and web app
│   ├── app\config.py                all settings (thresholds, sizes, paths); HELMET_* env overrides
│   ├── app\pipeline\                the AI pipeline (see section 8)
│   ├── app\services\                job running, file handling, reports
│   ├── app\api\                     FastAPI endpoints and background video jobs
│   ├── app\web\                     the web page (index.html, styles.css, app.js)
│   ├── app\cli.py                   command-line tool
│   ├── tests\                       pytest unit + end-to-end tests, sample files
│   ├── requirements.txt
│   └── README.md                    Phase 2 report (A/B tests, design decisions)
├── demo_dataset\                    categorised test images + answer_key.csv + README.txt
├── docs\figures\                    figures (e.g. fog_removal_demo.jpg)
├── start_app.bat                    starts the web app and opens the browser
├── demo.bat                         drag-and-drop demo without the browser
├── DEMO.md                          demo guide
└── PROJECT_REPORT.md                this report
Runtime folders: uploads\ · outputs\<run_id>\ · reports\<run_id>.json · logs\
```

Where each technique is implemented

Part	File : location
Dataset list and per-source class mapping	`phase1/src/data_prep.py:86` (`SOURCES`)
Cleaning thresholds (min size, head area, IoU limits, hashes)	`phase1/src/data_prep.py:65-84`
Downloading (Kaggle API, Roboflow REST API, Kaggle inputs)	`phase1/src/data_prep.py:175-279`
Label reading: YOLO txt and VOC XML	`phase1/src/data_prep.py:304-430`
Per-image cleaning (box clipping, tiny boxes, conflicts, perceptual hash)	`phase1/src/data_prep.py:447-525` (`clean_item`, `phash_dhash`)
Pseudo-labelling of rear-view heads (built; see section 13)	`phase1/src/data_prep.py:554-660`
Duplicate removal, scene groups, leak-free split	`phase1/src/data_prep.py:686-850` (`build_task`)
Offline weather augmentation (fog, rain, low light, motion blur, noise)	`phase1/src/data_prep.py:869-913`
Writing YOLO datasets and `manifest.csv`	`phase1/src/data_prep.py:916-976`
EDA and quality grids	`phase1/src/data_prep.py:1026-1117`
Training settings (model, image size, epochs, time limit, patience)	`phase1/src/train.py:68-80`
Online augmentation settings	`phase1/src/train.py:83-85`
Motorcycle-crop dataset for the plate model	`phase1/src/train.py:168` (`build_plate_crops`)
Training call (DDP, AMP, cosine LR, MuSGD)	`phase1/src/train.py:316-326`
Test evaluation, per-source evaluation	`phase1/src/train.py:359-410`
Deployment threshold (max-F1) and confusion matrix	`phase1/src/train.py:410-428`
Visual test, ONNX export	`phase1/src/train.py:451-486`
Image quality check and enhancement (DCP, CLAHE, gamma...)	`backend/app/pipeline/enhancement.py`
YOLO wrapper (predict, track)	`backend/app/pipeline/detectors.py`
Head → rider → motorcycle association, decision rule, duplicate-box removal, head-only fallback	`backend/app/pipeline/association.py`
Full per-image pipeline	`backend/app/pipeline/image_pipeline.py`
Video: sampling, tracking, voting, 4K handling	`backend/app/pipeline/video_pipeline.py`
OCR (PaddleOCR, flip/rotate retry, CPU-thread fix)	`backend/app/pipeline/ocr.py`
Indian plate format repair and validation	`backend/app/pipeline/plate_text.py`
Drawing boxes and labels	`backend/app/pipeline/annotate.py`
Reports, evidence crops, plate table	`backend/app/services/detection_service.py`
REST API	`backend/app/api/main.py`
Web page	`backend/app/web/`

5. Phase 1: data collection and preparation

5.1 Datasets used (11 public sources, each checked visually)

Source	Raw images	Used for	Main issue found and fixed
aryanvaid13 Indian Helmet (Kaggle)	942	helmet + plate	Class names were `0..4`, so the meaning was checked visually. 51 images had whole-body boxes labelled as faces and were removed.
anushkagovindkadam Smart Helmet (Kaggle)	485	helmet	Contained copies of another dataset; 67 duplicates removed.
aneesarom (Kaggle)	124	helmet + plate	RGBA images converted to RGB.
EdgeVision (Mendeley, 2026)	2,392	helmet	VOC XML labels converted to YOLO format.
bhumik Helmet & NumberPlate (Roboflow)	6,015	helmet + plate	Each image appeared 3× as augmented copies; about 1,224 unique remained.
miniproject HelmetDetection (Roboflow)	1,081	helmet + plate	Duplicates removed.
pkdarabi HelmetViolations (Kaggle)	1,937	plate	Helmet boxes cover the whole rider, so only plates were used; about 250 unique.
deepakat002 plates (Kaggle)	160	plate	Frames from 3 videos; each video kept within one split.
devgurucodes Traffic Violations (Kaggle)	11,772	plate	6,621 images had no plate labels and were dropped; 3,556 duplicates removed.
majorprojectssuu (Kaggle)	1,596	plate	Darknet labels with no names file; class ids checked visually.
Gujarat plates (Kaggle)	355	plate	–

Rejected: the Hugging Face "42k Indian helmet" set, whose repositories contain no images.

5.2 Design decisions about the labels

1. **Helmet labels are head boxes** (helmet / no_helmet). Sources that box the whole rider were not used for Model A.
2. **No trained "rider" class.** Sources disagree on what a rider box is, so motorcycles and persons come from the pretrained COCO model.
3. **Plates are detected on motorcycle crops**, not on the full image. A plate in a full traffic image is often only 20–40 px wide; cropping makes it about 5× larger and removes car plates.

5.3 Cleaning techniques (`data_prep.py:447-525`)

- Corrupt and tiny images (shorter side under 128 px) removed; very large images scaled to at most 1920 px.
- Boxes clipped to the image; boxes thinner than 4 px removed; same-class double labels merged (IoU > 0.9).
- **Label-error filters:** an image is removed if a "head" box covers more than 40 % of it, or if the same head is labelled both helmet and no_helmet (IoU > 0.6). A whole source is flagged if its median head box is larger than 10 % of the image (it boxes bodies, not heads).

5.4 Duplicate removal and leak-free split (`data_prep.py:686-850`)

- **Perceptual hashing:** pHash and dHash (64-bit, mirrored images included). Two images are duplicates when pHash ≤ 6 **and** dHash ≤ 10 **and** at least 60 % of their label boxes match. The box check matters: fixed-camera frames have identical hashes but show different riders, and those are kept.
- **Roboflow augmented copies** are grouped by their shared base file name.
- **Scene groups:** similar scenes (pHash ≤ 12) and frames from the same video are always placed in the **same split**, so the test set never contains a near-copy of a training image (no data leakage).
- **Stable split:** each group's split is decided by an MD5 hash of the group key (80 % train / 10 % val / 10 % test), so it does not change when data is added.
- **Indian-only test set:** test images come only from Indian sources.
- Background images (no boxes) capped at 10 %. Minority-class oversampling was built in but not needed (7,456 helmet vs 5,170 no-helmet boxes).

5.5 Augmentation

- **Offline, train only** (`data_prep.py:869-913`): 20 % of training images got one extra copy with a weather effect: fog, rain, low light, motion blur or sensor noise.
- **Online, during training** (`train.py:83-85`): mosaic, scale 0.5–1.5×, translation 10 %, rotation ±5°, shear 2°, perspective, HSV colour/brightness, mixup 0.1 (helmet only), horizontal flip (helmet only, because mirrored plate text is unrealistic), mosaic switched off for the last 10 epochs.

5.6 Final datasets

Dataset	Train	Val	Test	Total
Helmet	4,747 (+908 weather copies)	512	334	5,593
Plate (full images; crops built from these)	4,193	311	204	4,708

---

6. Phase 1: model training (`phase1/src/train.py`)

6.1 Why YOLO (YOLO26)

YOLO is a one-stage detector: one network pass predicts all boxes. YOLO26 is **NMS-free (end-to-end)**, which makes it faster and simpler to deploy. It was compared with alternatives:

Detector	Why not chosen
Faster R-CNN (two-stage)	5–10× slower, heavier deployment, no accuracy gain here
SSD	weak on small objects (heads and plates are small)
DETR / RT-DETR	needs more data and longer training; with about 7k images YOLO converges better

6.2 Training setup

Parameter	Value	Reason
Starting weights	YOLO26-m pretrained on COCO (transfer learning)	the "m" size is the best accuracy that fits a T4 GPU

Parameter	Value	Reason
Image size	960 px (helmet) / 640 px (plate crops)	heads are small, so 960 helps; crops are already enlarged
Batch	16 (8 per GPU, **DDP** on 2 GPUs)	fits 15 GB GPU memory with mixed precision (**AMP**)
Optimizer / LR	`auto` → **MuSGD**, lr 0.01, **cosine schedule**, 3 warm-up epochs	Ultralytics defaults tuned for YOLO26
Epochs	up to 200, capped at 10.5 h, **early stopping** after 40 epochs without improvement	fits Kaggle's 12 h session limit
Loss	box regression (L1 / IoU) + classification (BCE)	YOLO26 end-to-end loss
Evaluation	conf 0.001, IoU 0.7 for mAP; deployment threshold = **maximum-F1 point**	standard COCO-style evaluation
Seed	42	reproducible results
Hyper-parameter tuning	built but off (`RUN_TUNE = False`)	too expensive on the Kaggle GPU quota; the defaults are already tuned for YOLO26

6.3 Plate crops (`train.py:168`)

For Model B, the COCO detector finds every motorcycle in the plate dataset. Each motorcycle is cropped with **15 % padding**, and the plate labels are moved into crop coordinates. Plates whose motorcycle the detector missed get a **context crop** around the plate (train split only), so the model still learns from them. The same 15 % padding is used at run time.

6.4 Outputs of each training run

`metrics.json`, training curves, PR / F1 curves, confusion matrices (at conf 0.001 and at the deployment threshold), per-source test scores, visual test images, and `<task>_best.pt` plus ONNX export.

---

7. Phase 1: results

Model	Test mAP50	mAP50-95	Precision	Recall	F1	Deploy conf.
A: Helmet (YOLO26-m, 960 px)	**0.891**	0.531	0.838	0.840	0.839	0.40
B: Plate (YOLO26-m, 640 px crops)	**0.923**	0.482	0.910	0.890	0.900	0.37

Test sets: 334 Indian images (helmet) and 222 motorcycle crops from 204 Indian images (plate), never used for training or for choosing the best epoch.

Plate-model experiments

Run	Change	Test mAP50	Test recall
v1	full images at 960 px	0.653	0.726
v2	**motorcycle crops** at 640 px + context crops	0.768	0.735
v3 (final)	v2 + 3 more plate datasets (data nearly doubled)	**0.923**	**0.890**

Plate accuracy per test source: trafficviol 0.959, aryan (distant CCTV, hardest) 0.838, gujarat 0.995, anees 0.995, mpsuu 0.942.

Observations: the helmet model starts to overfit after about epoch 70 (validation classification loss rises), so more and harder data, rather than longer training, is the next lever. Distant CCTV views are the hardest domain for both models.

---

8. Phase 2: AI pipeline (`backend/app/pipeline/`)

8.1 Steps for one image (`image_pipeline.py`)

1. **Quality check** (`enhancement.py`): brightness, contrast, haze (dark channel), blur (Laplacian variance) and noise are measured. Thresholds were calibrated on 250 clean vs 80 degraded images per condition.
2. **Enhancement, only where a problem is found:**

Problem	Technique
Fog / haze	**Dark Channel Prior** dehazing with guided-filter refinement
Motion blur	**Unsharp masking**
Noise	**Non-local means** denoising (bilateral filter in fast video mode)
Low contrast	**CLAHE** (contrast-limited adaptive histogram equalisation) on the L channel
Low light	**Adaptive gamma correction**
Rain streaks	streak mask + **inpainting** (manual option, see 8.4)

3. **Detection:** the COCO model finds motorcycles and persons, and Model A finds helmet / no_helmet heads. Small images (≤ 960 px) run the COCO model at 640 px (its training size); larger images at 960 px. Overlapping duplicate motorcycle boxes are merged (IoU > 0.55, or > 85 % inside a stronger box).
4. **Association** (`association.py`): a person whose box overlaps a motorcycle's "rider zone" (the bike box extended upwards) becomes its rider, and a head is linked through its rider or by position above the bike. Heads not on any motorcycle (pedestrians) **never** cause a violation.
5. **Decision rule:** any `no_helmet` head on a bike → **VIOLATION** (driver *or pillion*, as required by MV Act Section 129); only `helmet` heads → **SAFE**; no head → **UNKNOWN**.
6. **Head-only fallback:** if no motorcycle is detected at all but heads are (close-ups, cropped photos), each rider is judged from the head, and the page says so.
7. **Plate:** for each VIOLATION, the motorcycle is cropped (+15 %), Model B finds the plate, and PaddleOCR reads it (`ocr.py`). If the read is not a valid number, the mirrored and 180°-rotated crops and the enhanced image are tried.
8. **Indian plate format repair** (`plate_text.py`): each position must be a letter or a digit, so look-alike characters are repaired (O↔0, I↔1, S↔5, B↔8, Z↔2, G↔6). A real state code is required. Supported formats: standard `MH12AB1234`, Delhi `DL3CAB1234`, Bharat series `22BH1234AA`. The "IND" mark and dealer text around the plate are ignored.
9. **Output:** annotated image (red = no helmet, green = helmet, yellow = plate), evidence crop per violation, plate crop, and `reports/<run_id>.json`.

8.2 Video (`video_pipeline.py`)

- 5 frames per second are analysed; the other frames reuse the last drawing, so the output keeps its original length and frame rate.
- **ByteTrack** gives each motorcycle a persistent ID. Each analysed frame casts a vote, and a motorcycle is reported only after **≥ 2 decided frames**, using the **majority** status. This removes single-frame errors and reports each rider once.
- Plates are read on up to 4 frames per motorcycle, and the most frequent valid text wins.
- 4K frames are scaled to Full HD for detection and the output video, while plates are read from the **full-resolution** frame.
- Output is an **H.264** MP4 that plays in any browser.

8.3 Does enhancement improve detection? (A/B test on 107 unseen images)

Condition	No enhancement	Auto enhancement
clean	0.845	–
fog	0.852	0.849
low light	**0.830**	0.805
motion blur	0.725	0.723
noise	0.832	0.834

Conclusion: because the models were trained with weather augmentation, enhancement does not raise detection accuracy, and it lowers it on low light. So **detection runs on the original image**, and the enhanced image is shown to the user and used as a second OCR attempt. **Exception: heavy fog.** Dehazing recovered a motorcycle lost at 70–85 % fog, so when fog is detected, detection uses the de-fogged image.

8.4 Important design decisions

Decision	Why
UNKNOWN instead of guessing	a wrong VIOLATION is worse than no decision
Plates read only for violators (option: all plates)	privacy, and speed
Rain removal is manual	the rain measure could not separate rainy from clear images (median 0.02 vs 0.03); applied to clear images it smeared detail
Plate on the motorcycle crop	matches training (65 % → 92 % mAP50)
All settings in one file (`config.py`)	reproducible; any value can be changed with a `HELMET_*` environment variable
GPU used automatically, CPU fallback	`backend/app/core/device.py`

---

9. Phase 3: web application (`backend/app/web/`, `backend/app/api/main.py`)

Scope (as decided): a simple page with no login and no database. The user uploads an image or video and sees the results on screen.

Features

- Drag-and-drop or browse upload (JPG, PNG, WEBP, BMP, MP4, AVI, MOV, MKV, WEBM; up to 500 MB) with an upload progress bar.
- Options: **Remove rain streaks**; **Read all number plates** (images).
- Video jobs run in the background, and the page shows live progress.
- Results: verdict banner; counts of motorcycles, violations, safe and unknown; processing time; enhancement note; **original vs processed** side by side (or the processed video); **violation table** with evidence photo, plate crop, plate number, "valid Indian format" / "check manually" tag, OCR confidence, rider count, and no-helmet confidence (or time range in video); **all-plates table**; download buttons for the processed file and the JSON report.

API endpoints

Method	Path	Result
GET	`/api/v1/health`	status, device, model errors
POST	`/api/v1/detect/image`	report (form fields `remove_rain`, `read_all_plates`)
POST	`/api/v1/detect/video`	`{job_id}`; processed in the background
GET	`/api/v1/jobs/{job_id}`	queued / running / done / failed, progress
GET	`/api/v1/results/{run_id}`	stored JSON report
GET	`/api/v1/files/{run_id}/{file}`	annotated media, evidence and plate crops

Start it with `start_app.bat`, which runs Uvicorn on `127.0.0.1:8000` and opens the browser when the models are loaded.

---

10. Testing and evaluation

10.1 Automated tests (`backend/tests/`)

25 tests pass: unit tests (plate text repair, association, duplicate-box removal, head-only fallback, enhancement) and end-to-end tests (images, video and API).

10.2 Demo test dataset (`demo_dataset/`, built by `phase1/tools/build_demo_dataset.py`)

- Images come **only from the held-out test split**. All of them were checked against the Kaggle training run's own manifests: **0 images seen in training** (10 that were found in the Kaggle training split were removed).
- `answer_key.csv` gives the true number of helmet and no-helmet heads (and plates) per image.

Folder	Content
01_no_helmet / 02_helmet_safe / 03_mixed	riders without helmets / all with helmets / both on one bike
04_crowded_traffic / 05_night_dark	6+ heads / real dark images
06_number_plates	plate test images
07–10 *_synthetic	test images with programmatically added fog, motion blur, rain, darkness

Folder	Content
11_real_world_new	new photos from Pexels, Unsplash and the team's own photos (no ground truth)
12_video	4 test videos (4K, Full HD, demo clip)

10.3 Full-system results (image-level verdict vs answer key)

Folder	Before pipeline fixes	Final
01_no_helmet	92 %	**100 %**
02_helmet_safe	56 %	**83 %**
03_mixed	96 %	**96 %**
04_crowded_traffic	68 %	**80 %**
05_night_dark	89 %	**100 %**
07_fog (synthetic)	92 %	**100 %**
08_motion_blur (synthetic)	50 %	**100 %**
09_rain (synthetic)	75 %	**92 %**
10_low_light (synthetic)	83 %	**100 %**
Total	**78 %**	**93 % (146 / 157)**
Images with no result	27	**0**

The final total includes one corrected dataset label: edgevision_02288 was labelled "2 helmets" in the original dataset, but manual checking showed both riders have none, and the model was right.

10.4 Number plates (folder 06, "Read all number plates" on)

	Before fix	Final
plate found	19 / 25	**24 / 25**
text read	11 / 25	**16 / 25**
valid Indian number	4 / 25	**11 / 25**

Clear plates are read correctly (e.g. GJ36AF2381, GJ03PM9162, GJ03MS8967). Most failures are plates only a few pixels high or blurred.

Plates under motion blur (6 clear plates, blur added):

Blur	Plate found	Text correct
none	6 / 6	6 / 6 (one with a single wrong letter)
7 px	4 / 6	0 / 6
11–15 px	4 / 6	0 / 6
21 px	2 / 6	0 / 6

Motion blur does not affect helmet detection (100 % on folder 08), and the plate is still located, but its text becomes unreadable even with light blur.

10.5 Videos (laptop CPU)

Video	Length	Processing	Result
demo clip (576×720)	4.4 s	31 s	1 violation, plate KL09AO3439 (real KL09AQ3439)
Full HD, 25 fps	7 s	55 s	5 motorcycles, all SAFE
4K, 30 fps	10.5 s	2 min 13 s	13 violations, 6 safe, plates DL13SX2607, DL5SA8697
4K, 60 fps	15 s	3 min 18 s (was 7 min 56 s)	10 violations, 3 safe, plate MP04JM7392

10.6 Manual testing (done by the team in the web app)

Each category was tested by uploading images and comparing with the answer key. The problems it revealed are listed in section 11 and were fixed.

---

11. Problems found and how they were solved

#	Problem	Cause	Solution
1	Duplicate false positives in data cleaning	fixed-camera frames have identical hashes but different riders	duplicate also requires ≥ 60 % matching boxes
2	Plate model only 65 % mAP50	plates tiny in full images	train on motorcycle crops, add 3 datasets → 92.3 %
3	CUDA out of memory in plate training	COCO detector still on the GPU	free GPU memory before training
4	Kaggle could not attach datasets in commit mode	kagglehub limitation	direct Kaggle API download URL
5	PaddleOCR crash on Windows	DLL conflict and oneDNN bug	import torch before paddle; `enable_mkldnn=False`
6	Enhancement lowered accuracy on low light	models already trained with weather augmentation	detect on the original; enhance only for fog
7	Rain detector could not separate rain from clear	weak rain metric	rain removal made a manual option
8	"0 motorcycles" on 27 of 157 test images	small images upscaled to 960 px for the COCO model	run the COCO model at 640 px on small images + head-only fallback
9	One scooter counted as 5 motorcycles	NMS-free model gave overlapping weak boxes	duplicate-box merging
10	Plates not shown for non-violators	by design, only violators' plates are read	"Read all number plates" option + whole-image search for close-ups
11	OCR read dealer text and the "IND" mark	extra text around the plate	search for a valid plate inside the text (exact matches only)
12	4K video took about 8 minutes	**PaddleOCR silently set PyTorch to 1 CPU thread** after its first call (2.4× slower), plus decoding 4K	restore the thread count after OCR; scale 4K to Full HD; read plates from the full frame → 2.4× faster

#	Problem	Cause	Solution
13	10 demo images had been in the Kaggle training split	local rebuild differed slightly from Kaggle	demo set filtered against the Kaggle manifests

12. Limitations

1. **Rear and side views of helmets** are sometimes classified as "no helmet" (most of the remaining errors in 02_helmet_safe, 83 %).
2. **Caps and other head coverings** can be mistaken for helmets, especially in small images.
3. **Dense crowds:** with 8–13 tiny heads, one mistake changes the image verdict (80 % in 04_crowded_traffic).
4. **Hidden motorcycles:** a bike mostly hidden by another vehicle or the image edge may not be detected, so its rider is linked to the neighbouring bike. The helmet status is still correct; only the motorcycle count is lower.
5. **Plate reading needs sharp, large plates.** Blur and plates a few pixels high give "unreadable". Similar letters (W/N, Q/O) can be confused on dirty plates. The system never invents a number, and a human can check the plate crop.
6. **Video tracking:** when a rider is hidden for a moment, the tracker can give them a new ID, so one rider can be counted twice.
7. **Speed on CPU:** about 13 s per second of 4K video on this laptop; an NVIDIA GPU is about 10× faster.
8. **Weather tests are synthetic** (fog, rain, blur and low light added by a program); a test on real rainy CCTV footage is future work.
9. **Pillion / driver rule:** the system follows the law (every rider must wear a helmet); a "driver-only" rule would need driver identification.

13. Future work

1. **Rear-view retraining:** the pseudo-labelling code for rear-view heads is already built (`data_prep.py:554-660`). The data-prep run must be done in an interactive Kaggle session, followed by a retrain. The model is replaced only if it beats 0.891.
2. **Hard negatives:** add caps, turbans and dupattas labelled as `no_helmet`.
3. **Owner notification:** a vehicle-registry database, human confirmation, then e-mail or SMS to the owner.
4. **History and dashboard:** a database of violations with search and statistics.
5. **Automatic rain detection** with a trained rain / no-rain classifier (the code has one switch point, `QualityReport.is_rainy`).
6. **Plate super-resolution** (e.g. Real-ESRGAN) for slightly blurred plates, always with human verification.
7. **Deployment:** Docker, a GPU server or an NVIDIA Jetson edge device at the junction; real CCTV testing.
8. **Configurable rules**, e.g. driver-only, triple riding.

14. How to run the project

Task	How
Web app	double-click `start_app.bat` → browser opens http://localhost:8000
Drag-and-drop demo	drag an image or video onto `demo.bat`
Command line	`cd backend` → `..\.venv\Scripts\python -m app.cli <file or folder> [--remove-rain] [--open]`
Tests	`cd backend` → `..\.venv\Scripts\python -m pytest -q`
Re-train (Kaggle)	`phase1/notebooks/01_data_prep.ipynb` (CPU, internet on, `ROBOFLOW_API_KEY` secret, EdgeVision input) → `02_train.ipynb` (GPU T4 ×2, notebook 1 output attached, `TASK = "helmet"` or `"plate_crop"`)
Rebuild notebooks	`python phase1/tools/build_notebooks.py`
Rebuild demo set	`python phase1/tools/build_demo_dataset.py <datasets_dir> demo_dataset --per 25 --kaggle-manifests <folder>`

15. Glossary

Term	Meaning
mAP50	mean Average Precision at 50 % box overlap: the standard detection accuracy score
mAP50-95	the same, averaged over stricter overlaps (50–95 %): how tight the boxes are
Precision / Recall / F1	share of detections that are correct / share of real objects found / their harmonic mean
IoU	Intersection over Union: overlap of two boxes
NMS	Non-Maximum Suppression: removing duplicate boxes (YOLO26 does not need it)
Transfer learning	starting from a model pretrained on a large dataset (COCO)
DDP	Distributed Data Parallel: training on 2 GPUs at once
AMP	Automatic Mixed Precision: faster training with 16-bit maths
pHash / dHash	perceptual image hashes used to find near-duplicate images
Data leakage	test images that are (near) copies of training images, giving falsely high scores
Dark Channel Prior	a fog-removal method based on the darkest colour channel in local patches
CLAHE	contrast-limited adaptive histogram equalisation
ByteTrack	a multi-object tracker that keeps object IDs across video frames
OCR	Optical Character Recognition: reading text from an image
Pseudo-labelling	labelling unlabelled data with a trained model's confident predictions